

CS 452/652 Winter 2025 - Lecture 5

ABI, Coding, Kernel

Jan 21, 2025 [prev](#)

Application Binary Interface (ABI)

- subroutine call is a simple context switch (new stack frame)
 - coroutine call is similar to task switch (but deterministic)
- ABI standardized to support separate compilation units
 - some hardware specifications, mostly software/compiler convention
 - stack pointer must be 16-byte aligned
 - important (for us): registers preserved during function call (caller-owned, callee-saved)
- argument passing: registers and stack, stack alignment (16)
 - x0-x7 arguments, return x0/x1 (caller-saved)
 - x8 return ptr (instead of x0) for structs > 16 bytes by value
 - set up by caller, register itself caller-saved
 - x9-x15 local registers (caller-saved)
 - x16,x17 temporary for linker (caller-saved)
 - x18 platform (thread-local storage?) (caller-saved)
 - x19-x28 global registers (callee-saved)
 - x29 frame pointer (callee-saved)
 - x30 link register (return address, callee-saved)
 - condition codes (pstate): undefined on entry/return of subroutine
- floating point registers: do not use or self-study

→ can use ABI rules when implementing (synchronous) system call!

Coding: Mixing C/C++ and Assembler

See various examples file in [demo05](#). Take a look at the compiled .s source

- main.c, adder.c - basic argument passing in x0, x1
 - main.c, asm_adder.S - manual assembler routine working with C caller
 - adder10.c - passing more than 8 arguments (on stack)
 - main3.c, data_adder.c - memory-based data exchange between C and assembler
 - asm.c - embed assembler in C code; exchange data with local variables
- special register access: msr/mrs

Recommendations

- use embedded assembler via asm only for short code sequences that do not access stack
- use assembler routine linked as C routine for context-switch code

Context Switch (details)

- After SVC
 - handler in exception vector executes
 - processor in EL1
 - ELR_EL1 holds return address

- ESR_EL1 contains N from SVC
- SPSR holds the processor state from before SVC
- SP is banked SP_EL1 (run with SPSel set to 1)
- C/C++ function call: compiler has saved x0-x18 (as needed)
- SVC Exception Handler
 - must save ELR (and later SPSR for IRQ)
 - access to SP_EL0 direct or via SPSel = 0 (temporarily)
 - has access to x0-x30
 - choices for storing user context:
 - on user stack, or
 - in task descriptor
 - then restore kernel context
 - symmetry with kernel entry and exit (syscall vs irq)?
- Kernel Exit
 - store kernel context (function call, ABI)
 - restore given user context (next task to be run)
 - return to user mode via eret
- System Call Parameters
 - example: `int Create(int priority, void (*function)())`
 - how does the kernel receive the two parameters?
 - how does it return the int?
 - simple answer: can use the same arg/return conventions established by the ABI
 - user task puts params into x0 and x1 before svc
 - exception handler saves application context
 - kernel can inspect saved x0 and x1 to find parameters
 - kernel overwrites saved x0 with return value before eret
 - after eret, user task looks in x0 to find result
 - system call stubs: create a user-level function corresponding to each system call
 - compiler will have placed params in x0 and x1 prior to branching to Create
 - compiler expects return value in x0 on ret from Create

```
int Create(int priority, void (*function)()) {
    --> svc N
}
```

Task Creation

- task termination: `Exit()` system call → kernel retires task
- end of user routine? use dedicate start routine
or add equivalent functionality to the fake initial user stack

```
void task_start(void (*task_function)()) {
    task_function();
    Exit();
}
```

Kernel Initialization

- initially in boot.S:
 - sets up EL1 stack pointer, masks interrupts, jumps to kmain
- what else needs to happen?
 - need to initialize exception vector

- need to initialize any kernel data structures
 - allocate slab of task descriptors and stacks and manage via free list
- need to create an initial task, and context switch to it
- initial task will need to run some user-level function, at some priority
 - hard code function pointer into kernel
 - set up new task context to be the "saved" context for user task
 - return from exception (restore user context, then `eret`)

Additional Information

An earlier context-switch document by Bill Cowan is available [here](#). This is certainly not the only way to write a context switch and I do not necessarily recommend (or not recommend) this particular approach, but I figure every bit of information can help.